10. Testing

Detailed beginner handbook - English and Arabic

Technologies: Vitest, Pytest

What you will learn

Write an automated check for a calculation.

Tests document expected behaviour and catch regressions before users do.

الهدف: بناء مثال صغير وفهمه خطوة بخطوة.

شرح بسيط: تعلم الفكرة أولا، ثم جرب المثال الصغير ببيانات اصطناعية فقط.

How to study

Study one lesson at a time. Type examples yourself, predict the result, run the code, then change one thing. Keep a notebook of new words, errors, root causes, and fixes.

Course map

Setup and mental model; ten core concepts; a guided mini-project; the real dashboard architecture; debugging; exercises and answers.

Lesson 1 - Setup and mental model

Prerequisites

Windows 10 or 11, VS Code, PowerShell, and permission to install learning dependencies. Use only synthetic data in tutorials and screenshots.

Setup sequence

1. Open PowerShell.
2. Go to learning-examples/10-testing.
3. Read README.md.
4. Run pip install -r requirements.txt; pytest.
5. Read the complete error if it fails.
6. Change one safe value and rerun.

الخطوات: افتح مجلد المثال، شغل الأمر، غير قيمة واحدة، ولاحظ النتيجة.

Mental model

Treat Vitest, Pytest as one layer in a system. Identify the input, the transformation or rule, the output, and possible failures. Understanding that flow is more valuable than memorising syntax.

Checkpoint

Explain: What problem does this technology solve? What is its input? What output should it produce? What can fail?

Lesson 2 - Why tests matter and what a useful test proves

What is it?

A useful test protects an important behaviour with deterministic input and a clear expected result. It should fail for the bug and pass for the correct implementation.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In Testing, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
def total(values): return sum(values)

def test_total_adds_values():
    assert total([4,8]) == 12
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Why tests matter and what a useful test proves.

هذا الدرس يشرح: Why tests matter and what a useful test proves. اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك.

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 3 - Arrange, Act, Assert and descriptive test names

What is it?

Arrange creates input, Act calls the subject, and Assert compares the outcome. A name should state the scenario and expected behaviour.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In Testing, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
def test_percentage():
    # Arrange
    part,whole=3,12
    # Act
    result=part/whole*100
    # Assert
    assert result==25
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Arrange, Act, Assert and descriptive test names.

اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك. Arrange, Act, Assert and descriptive test names هذا الدرس يشرح:

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 4 - Pytest discovery, assertions, fixtures, and parametrization

What is it?

Pytest discovers test_ files and test_ functions. Plain assert gives readable diffs; fixtures provide setup; parametrize checks many cases with one test body.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In Testing, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
import pytest
@pytest.mark.parametrize('values,expected',[([],0),([2],2),([2,3],5)])
def test_total(values,expected): assert sum(values)==expected
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Pytest discovery, assertions, fixtures, and parametrization.

هذا الدرس يشرح: Pytest discovery, assertions, fixtures, and parametrization. اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك.

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 5 - Vitest suites, expectations, modules, and mocking

What is it?

Vitest groups tests with describe, defines them with test, and checks results with expect. Mock modules only at real external boundaries.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In Testing, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
import {expect,test} from 'vitest';
import {total} from './math';
test('adds values',()=>expect(total([4,8])).toBe(12));
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Vitest suites, expectations, modules, and mocking.

اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك. Vitest suites, expectations, modules, and mocking هذا الدرس يشرح:

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 6 - Unit, integration, API, UI, and end-to-end test boundaries

What is it?

Unit tests isolate logic; integration tests connect modules; API tests exercise routes; UI tests exercise visible behaviour; end-to-end tests cover complete user flows.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In Testing, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
# Unit: total([4,8])
# Integration: workbook -> calculation
# API: POST /query
# UI: choose month and see total
# E2E: upload -> chart -> export
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Unit, integration, API, UI, and end-to-end test boundaries.

هذا الدرس يشرح: Unit, integration, API, UI, and end-to-end test boundaries. اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك.

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 7 - Testing success, empty, invalid, and failure cases

What is it?

Test typical success plus empty collections, missing fields, malformed values, duplicates, boundary dates, permission errors, and failures from dependencies.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In Testing, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
def test_empty(): assert total([])==0
def test_invalid():
    with pytest.raises(TypeError): total(None)
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Testing success, empty, invalid, and failure cases.

هذا الدرس يشرح: Testing success, empty, invalid, and failure cases. اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك.

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 8 - Temporary files and synthetic test data

What is it?

Temporary directories isolate generated files and are cleaned automatically. Synthetic factories create realistic shapes without copying production personal data.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In Testing, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
def test_export(tmp_path):
    target=tmp_path/'report.txt'
    target.write_text('Synthetic',encoding='utf-8')
    assert target.read_text()=='Synthetic'
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Temporary files and synthetic test data.

اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك. Temporary files and synthetic test data هذا الدرس يشرح:

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 9 - Mocking time, network calls, and external services carefully

What is it?

Freeze time when dates affect results. Mock network responses for deterministic failures. Excessive mocking can produce tests that pass while integration is broken.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In Testing, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
from unittest.mock import patch
@patch('service.fetch')
def test_network_failure(fetch):
    fetch.side_effect=TimeoutError()
    assert service.load()=={'error':'timeout'}
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Mocking time, network calls, and external services carefully.

اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك. Mocking time, network calls, and external services carefully هذا الدرس يشرح:

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 10 - Coverage, regression tests, and avoiding brittle tests

What is it?

Coverage locates unexecuted code but does not prove correctness. Add a regression test for every fixed bug and avoid assertions tied to irrelevant implementation details.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In Testing, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
# Coverage finds unexecuted lines; it does not prove assertions are meaningful.
# Add a regression test that fails before each bug fix.
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Coverage, regression tests, and avoiding brittle tests.

هذا الدرس يشرح: Coverage, regression tests, and avoiding brittle tests. اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك.

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 11 - Reading failures and debugging the smallest reproducible case

What is it?

Read the first failure, reproduce only that test, inspect the actual value, reduce the input, and distinguish a product defect from an outdated expectation.

Why do we need it?

Without understanding this idea, code may appear to work while handling data incorrectly or failing on a different input. In Testing, this concept helps create behaviour that is explicit, testable, and easier to change.

Syntax and example

```
pytest -q test_math.py::test_total
# Read expected versus actual, inspect the value, then reduce the input.
```

What happens step by step

1. Read the example from the first line downward.
2. Identify the input or starting value.
3. Identify the operation, rule, or declaration.
4. Find the output or visible effect.
5. Predict the result before running it.
6. Run it and compare the real result with your prediction.

Try it yourself

Type the example instead of pasting it. Change one name and one synthetic value. Then introduce an empty or incorrect value and observe the result. Explain how this demonstrates Reading failures and debugging the smallest reproducible case.

هذا الدرس يشرح: Reading failures and debugging the smallest reproducible case. اكتب المثال بنفسك، حدد المدخلات والعملية والنتيجة، ثم غير قيمة واحدة لتفهم السلوك.

Quick check

Can you define the concept, point to it in the example, predict the output, and change the example without help? If not, repeat this lesson before continuing.

Lesson 7 - Guided mini-project

Project goal

Write an automated check for a calculation.

Starting example

```
def total(values): return sum(values)
def test_total():
    assert total([2,3]) == 5
```

Read the code

Find imported tools or page elements. Identify the synthetic input. Find the function, event, route, or transformation that changes it. Finally, locate where the result is displayed, returned, saved, or packaged.

Build sequence

1. Run the untouched example.
2. Record the result.
3. Rename one unclear value.
4. Add a synthetic record.
5. Validate empty input.
6. Validate incorrect input.
7. Add a helpful error.
8. Rerun every case.
9. Compare with exercise-solution.
10. Explain every line.

Definition of done

Normal, empty, and invalid cases behave deliberately; no private data is used; and you can explain every line without reading the guide.

Lesson 8 - The real dashboard

Architecture connection

The application combines Vitest, Pytest with the rest of the stack. The frontend gathers filters and files; the local API validates requests; the data layer transforms workbook rows; charts present aggregates; export tools create files; and the desktop layer packages the application for Windows.

Trace the upload feature

The user selects a workbook. The frontend sends it to the local API. The backend validates the file and sheets. The data layer creates safe tables. The API returns metadata. React updates state. Plotly displays results. Identify exactly where this guide's technology participates.

Privacy and quality

Do not place narrative, contact, or identifying fields in tutorials, logs, screenshots, tests, or exports. Use generated identifiers and synthetic totals. Validate at each boundary and collect only fields required for the calculation.

Architecture exercise

Draw five boxes: UI, API, Data Processing, Export, Desktop Packaging. Add arrows and label the data. Circle the box related to this guide and add two validation checks.

Lesson 9 - Debugging

Reliable debugging loop

1. Reproduce with the smallest input.
2. Read the first meaningful error.
3. Find the exact file and line.
4. Inspect value and type.
5. Compare expected and actual results.
6. Change one thing.
7. Rerun the same case.
8. Add a regression test.

Common beginner mistakes

- Wrong folder or command.
- Missing dependency or inactive environment.
- Misspelled file, variable, field, route, or import.
- Assuming input is always present.
- Mixing setup, processing, and display in one block.
- Hiding an exception rather than understanding it.
- Using real sensitive data in a test.

أخطاء شائعة: تشغيل الأمر من مجلد خاطئ، نسيان تثبيت التبعيات، أو تغيير أشياء كثيرة بمرة واحدة.

Error notebook

Record: command, expected result, actual error, root cause, smallest fix, and test added. This turns errors into reusable knowledge.

Lesson 10 - Exercises and answers

Exercises

1. Explain the technology in three sentences.
2. Recreate the example without copying.
3. Add one normal synthetic record.
4. Handle empty input.
5. Handle invalid input.
6. Improve unclear names.
7. Add or describe one test.
8. Locate the technology in the dashboard.
9. Record one error and root cause.
10. Add one mini-project feature.

تمرين: أضف شهرا أو قيمة آمنة جديدة، ثم اشرح ما حدث بكلماتك.

Answer guide

A strong solution is observable and explainable: the documented command works; output changes for the new record; empty and invalid cases have deliberate results; names communicate purpose; a test states an expected result; and the architecture explanation identifies the correct boundary.

Next step	Next: 11. PNG and PDF Export